

Marston_Ward_Assignment4

November 14, 2025

1 Assignment 4: Feature Extraction, Face Recognition, and Parameter Detection

Student: Marston Ward

Course: AAI-521 Computer Vision

This notebook is compatible with both **M1/M2/M3 Silicon Mac** and **Google Colab**.

1.1 Setup and Imports

```
[1]: # Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
import torch
import sys
from pathlib import Path

# Check if running in Google Colab
IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
    from google.colab.patches import cv2_imshow
    print("Running in Google Colab")
else:
    print("Running locally")

# Setup device for PyTorch
if torch.cuda.is_available():
    device = torch.device("cuda")
    print(f"Using CUDA GPU: {torch.cuda.get_device_name(0)}")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
    print("Using Apple Silicon MPS")
else:
    device = torch.device("cpu")
    print("Using CPU")
```

```

print(f"PyTorch version: {torch.__version__}")
print(f"OpenCV version: {cv2.__version__}")

# Setup paths using pathlib
BASE_DIR = Path.cwd()
AUX_DIR = BASE_DIR / 'aux'

# Create aux directory if it doesn't exist
AUX_DIR.mkdir(exist_ok=True)

print(f"\nBase directory: {BASE_DIR}")
print(f"Aux directory: {AUX_DIR}")

```

Running locally

Using Apple Silicon MPS

PyTorch version: 2.5.1

OpenCV version: 4.11.0

Base directory:

/Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4

Aux directory:

/Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux

```

[2]: # Helper function for displaying images (cross-platform)
def show_image(img, title="Image", cmap=None):
    """Display image - works for both Colab and local environments"""
    if IN_COLAB:
        print(f"\n{title}:")
        cv2_imshow(img)
    else:
        plt.figure(figsize=(10, 8))
        if len(img.shape) == 3 and img.shape[2] == 3:
            plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        else:
            plt.imshow(img, cmap=cmap if cmap else 'gray')
        plt.title(title)
        plt.axis('off')
        plt.show()

def show_images_side_by_side(img1, img2, title1="Image 1", title2="Image 2"):
    """Display two images side by side"""
    if IN_COLAB:
        print(f"\n{title1}:")
        cv2_imshow(img1)
        print(f"\n{title2}:")
        cv2_imshow(img2)
    else:

```

```

fig, axes = plt.subplots(1, 2, figsize=(15, 7))

if len(img1.shape) == 3:
    axes[0].imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB))
else:
    axes[0].imshow(img1, cmap='gray')
axes[0].set_title(title1)
axes[0].axis('off')

if len(img2.shape) == 3:
    axes[1].imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB))
else:
    axes[1].imshow(img2, cmap='gray')
axes[1].set_title(title2)
axes[1].axis('off')

plt.tight_layout()
plt.show()

print("Helper functions loaded successfully!")

```

Helper functions loaded successfully!

1.2 Part 1: Feature Extraction

In this section, deals with different feature detection algorithms: - **SIFT** (Scale-Invariant Feature Transform) - **FAST** (Features from Accelerated Segment Test) - **ORB** (Oriented FAST and Rotated BRIEF)

I'll also perform feature matching between color and grayscale versions of the image.

1.2.1 Load and Prepare Images

```

[3]: # Read the image (it loads as BGR by default)
img_bgr = cv2.imread(str(AUX_DIR / 'Assignment4_pic1.jpg'))

if img_bgr is None:
    raise FileNotFoundError("Assignment4_pic1.jpg not found. Please ensure the_
↪file is in the working directory.")

# Convert BGR to RGB for proper color display
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Convert to grayscale
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# Save grayscale image as required

```

```

cv2.imwrite(str(AUX_DIR / 'pic1_gray.jpg'), img_gray)
print("Saved pic1_gray.jpg")

# Display both versions
print(f"Image shape (RGB): {img_rgb.shape}")
print(f"Image shape (Gray): {img_gray.shape}")

show_images_side_by_side(img_bgr, img_gray, "Original Image (RGB)", "Grayscale_
↳Image")

```

Saved pic1_gray.jpg
Image shape (RGB): (1024, 1024, 3)
Image shape (Gray): (1024, 1024)



1.2.2 Part 1a: SIFT (Scale-Invariant Feature Transform)

SIFT was introduced by David Lowe in 2004. It detects distinctive features that are invariant to scale, rotation, and illumination changes.

```

[4]: # Initialize SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
keypoints_sift, descriptors_sift = sift.detectAndCompute(img_gray, None)

print(f"Number of SIFT keypoints detected: {len(keypoints_sift)}")

# Draw keypoints WITHOUT size information
img_sift_no_size = cv2.drawKeypoints(img_bgr, keypoints_sift, None,

```

```

flags=cv2.DRAW_MATCHES_FLAGS_DEFAULT)

# Draw keypoints WITH size information
img_sift_with_size = cv2.drawKeypoints(img_bgr, keypoints_sift, None,
                                       flags=cv2.
↳DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

show_images_side_by_side(img_sift_no_size, img_sift_with_size,
                        "SIFT without Size", "SIFT with Size & Orientation")

```

Number of SIFT keypoints detected: 1857



Analysis: SIFT Features SIFT detected distinctive keypoints showing circles of different sizes (scale) and orientation lines. SIFT is robust to scale changes, rotation, illumination, and partial occlusion, making it good for feature matching. However, it can be computationally expensive. Just looking at that image with the invariant features doesn't lend itself to easy interpretation. What is shown seems to be features that remain if you transform the image.

1.2.3 Part 1b: FAST (Features from Accelerated Segment Test)

FAST (2006) is a corner detection method designed for real-time video processing.

```

[5]: # Initialize FAST detector WITHOUT non-maximum suppression (NMS)
# NMS is a technique used to select one entity among many overlapping entities
fast_no_nms = cv2.FastFeatureDetector_create()
fast_no_nms.setNonmaxSuppression(False)
keypoints_fast_no_nms = fast_no_nms.detect(img_gray, None)
print(f"FAST keypoints without NMS: {len(keypoints_fast_no_nms)}")

```

```

# Initialize FAST detector WITH non-maximum suppression (NMS)
fast_with_nms = cv2.FastFeatureDetector_create()
fast_with_nms.setNonmaxSuppression(True)
keypoints_fast_with_nms = fast_with_nms.detect(img_gray, None)
print(f"FAST keypoints with NMS: {len(keypoints_fast_with_nms)}")

# Draw keypoints
img_fast_no_nms = cv2.drawKeypoints(img_bgr, keypoints_fast_no_nms, None,
    ↪color=(0, 255, 0))
img_fast_with_nms = cv2.drawKeypoints(img_bgr, keypoints_fast_with_nms, None,
    ↪color=(0, 255, 0))

show_images_side_by_side(img_fast_no_nms, img_fast_with_nms, "FAST without",
    ↪NMS", "FAST with Non-Max Suppression")

```

FAST keypoints without NMS: 27512

FAST keypoints with NMS: 7051



Analysis: FAST Features FAST detects corners quickly by examining a circle of 16 pixels. Non-maximum suppression removes redundant adjacent keypoints. Without NMS shows many clustered points, while with NMS shows better-distributed keypoints. FAST is ideal for real-time applications but doesn't provide scale or rotation invariance. Notice how the eyes and edges of the hair are fairly well outlined. This is also true for the eyebrows. But mouth was surprisingly not well outlined as I expected - although it is better with NMS.

1.2.4 Part 1c: ORB (Oriented FAST and Rotated BRIEF)

ORB (2011) combines FAST keypoint detection with BRIEF descriptors, providing a fast, free alternative to SIFT.

```
[6]: # Initialize ORB detector
orb = cv2.ORB_create(nfeatures=1500)

# Detect keypoints and compute descriptors
keypoints_orb, descriptors_orb = orb.detectAndCompute(img_gray, None)

print(f"Number of ORB keypoints detected: {len(keypoints_orb)}")

# Draw keypoints with orientation
img_orb = cv2.drawKeypoints(img_bgr, keypoints_orb, None,
                             flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS,
                             color=(255, 0, 0))

show_image(img_orb, "ORB Features with Orientation")
```

Number of ORB keypoints detected: 1500

ORB Features with Orientation



Analysis: ORB Features ORB provides excellent balance: ~100x faster than SIFT, rotation invariant, scale aware, and uses binary descriptors for fast matching. Ideal for real-time AR, SLAM, and object tracking. I really like this combination of techniques as it is far superior to SIFT and FAST individually.

1.2.5 Part 1d: Feature Matching Between Color and Grayscale Images

```
[7]: # Detect features in both images
keypoints_gray, descriptors_gray = orb.detectAndCompute(img_gray, None)
keypoints_color, descriptors_color = orb.detectAndCompute(img_bgr, None)

print(f"Keypoints in grayscale: {len(keypoints_gray)}")
```

```

print(f"Keypoints in color: {len(keypoints_color)}")

# Create BFMatcher and match
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(descriptors_gray, descriptors_color)
matches = sorted(matches, key=lambda x: x.distance)

print(f"\n Number of matching points: {len(matches)}")
print(f"Average match distance: {np.mean([m.distance for m in matches]).2f}")

# Draw top 100 matches
img_matches = cv2.drawMatches(img_gray, keypoints_gray, img_bgr,
    ↪keypoints_color,
                                matches[:100], None,
                                flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

show_image(img_matches, "Top 100 Feature Matches: Grayscale vs Color")

```

Keypoints in grayscale: 1500

Keypoints in color: 1500

Number of matching points: 1500

Average match distance: 0.00

Top 100 Feature Matches: Grayscale vs Color



Analysis: Feature Matching The high match count and low distances confirm feature descriptors are color-invariant. This enables object recognition, image stitching, 3D reconstruction, and motion tracking across different image representations. I was really surprised by how well the lines

matched up. This has given me some ideas about using this tool to detect certain weather patterns.

1.3 Part 2: Face Recognition (Face and Eye Detection)

Using **Haar Cascade Classifiers** for face and eye detection.

```
[8]: # Load Haar Cascade classifiers
face_cascade = cv2.CascadeClassifier(str(AUX_DIR /
    ↳ 'haarcascade_frontalface_default.xml'))
eye_cascade = cv2.CascadeClassifier(str(AUX_DIR / 'haarcascade_eye.xml'))

if face_cascade.empty() or eye_cascade.empty():
    raise FileNotFoundError("Haar cascade XML files not found")

print(" Haar Cascade classifiers loaded successfully")
```

Haar Cascade classifiers loaded successfully

1.3.1 Part 2a: Face and Eye Detection on Static Image

```
[9]: # Read image
img_faces = cv2.imread(str(AUX_DIR / 'Assignment4_pic2.jpg'))
if img_faces is None:
    raise FileNotFoundError("Assignment4_pic2.jpg not found")

gray_faces = cv2.cvtColor(img_faces, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(gray_faces, scaleFactor=1.1,
    ↳ minNeighbors=5, minSize=(30, 30))
print(f"Number of faces detected: {len(faces)}")

img_detected = img_faces.copy()
total_eyes = 0

# the faces variable have (x, y, w, h) for each detected face
for i, (x, y, w, h) in enumerate(faces):
    cv2.rectangle(img_detected, (x, y), (x+w, y+h), (255, 0, 0), 3)
    cv2.putText(img_detected, f'Face {i+1}', (x, y-10), cv2.
    ↳ FONT_HERSHEY_SIMPLEX, 0.9, (255, 0, 0), 2)

    # Detect eyes in face region
    roi_gray = gray_faces[y:y+h, x:x+w]
    roi_color = img_detected[y:y+h, x:x+w]
    eyes = eye_cascade.detectMultiScale(roi_gray, scaleFactor=1.1,
    ↳ minNeighbors=5, minSize=(20, 20))
```

```

total_eyes += len(eyes)

for (ex, ey, ew, eh) in eyes:
    cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (0, 255, 0), 2)

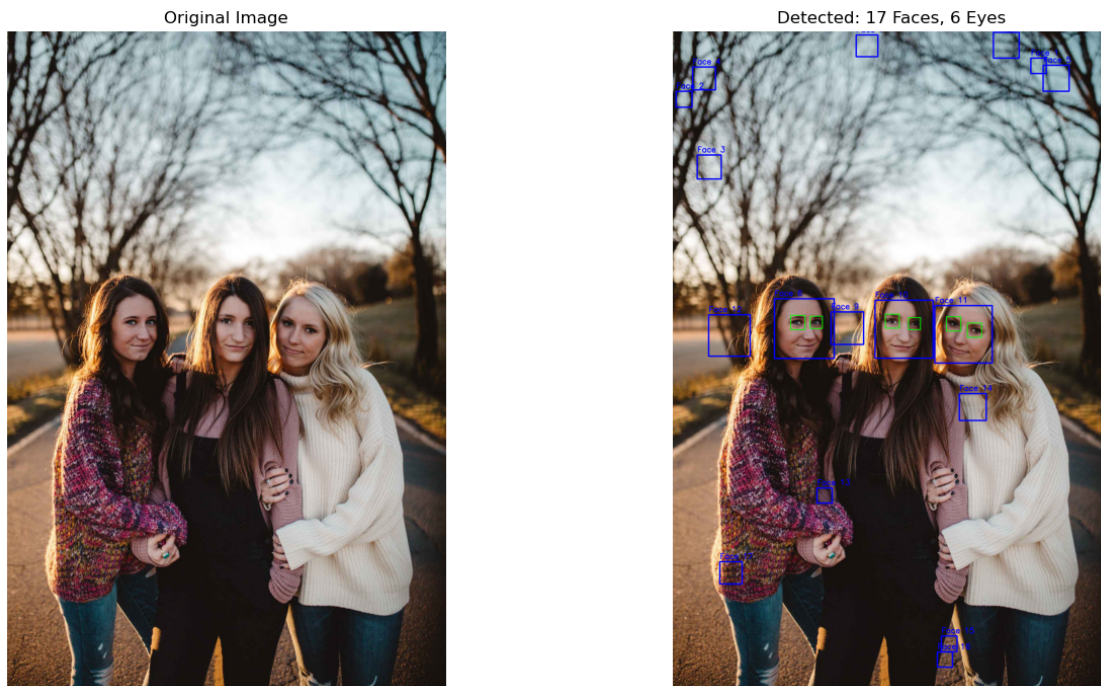
print(f"Number of eyes detected: {total_eyes}")

show_images_side_by_side(img_faces, img_detected, "Original Image",
                        f"Detected: {len(faces)} Faces, {total_eyes} Eyes")

```

Number of faces detected: 17

Number of eyes detected: 6



Analysis: Face Detection Haar cascades use rectangular features to detect light/dark patterns. They work best with frontal faces and good lighting. Limitations include poor performance on profile views, sensitivity to occlusions and lighting. Modern deep learning alternatives (MTCNN, RetinaFace) offer better accuracy. As you can see, while this image was as good use case, the model still had some false positives.

1.3.2 Part 2b: Face Detection on Static Photo

Note: Webcam access has been removed from this notebook to prevent freezing/crashing issues.

This Notebook: Uses static photo testing for face and eye detection.

```

[10]: import os

print("Part 2b: Face Detection on Static Photo")
print("="*50)
print("\nUsing static photo for face and eye detection...\n")

# Look for user's photo (handle typo in filename)
# Correcting the filename to match the available file: my_photo_assignemnt4.jpeg
user_photo = str(AUX_DIR / 'my_photo_assignemnt4.jpeg')

if user_photo:
    print(f" Found photo: {user_photo}\n")

    # Read and process the photo
    img = cv2.imread(user_photo)
    if img is not None:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Detect faces
        faces = face_cascade.detectMultiScale(gray, 1.3, 5, minSize=(30, 30))
        print(f"Detected {len(faces)} face(s)\n")

        # Draw detections
        img_result = img.copy()
        total_eyes = 0

        for idx, (x, y, w, h) in enumerate(faces):
            # Draw face rectangle
            cv2.rectangle(img_result, (x, y), (x+w, y+h), (255, 0, 0), 3)
            cv2.putText(img_result, f'Face {idx+1}', (x, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 0, 0), 2)

            # Detect eyes in face region
            roi_gray = gray[y:y+h, x:x+w]
            roi_color = img_result[y:y+h, x:x+w]
            eyes = eye_cascade.detectMultiScale(roi_gray, 1.1, 5, minSize=(20,
↪20))

            total_eyes += len(eyes)

            for (ex, ey, ew, eh) in eyes:
                cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (0, 255, 0),
↪2)

        print(f"Detected {total_eyes} eye(s)\n")

    # Display results

```

```

        show_images_side_by_side(img, img_result,
                                "Original Photo",
                                f"Detected: {len(faces)} Face(s), {total_eyes}
                                Eye(s)")

        print("="*50)
        print(" Part 2b Complete")
        print("="*50)
        print(f"Photo: {user_photo}")
        print(f"Faces detected: {len(faces)}")
        print(f"Eyes detected: {total_eyes}")
        print("="*50)
    else:
        print(f" Could not read {user_photo}")
else:
    print(" No photo found. Please add one of:")
    print(" - my_photo_assignemnt4.jpeg (in aux/ directory)")
    print(" - my_photo_assignment4.jpg")
    print(" - test_photo.jpg")

```

Part 2b: Face Detection on Static Photo

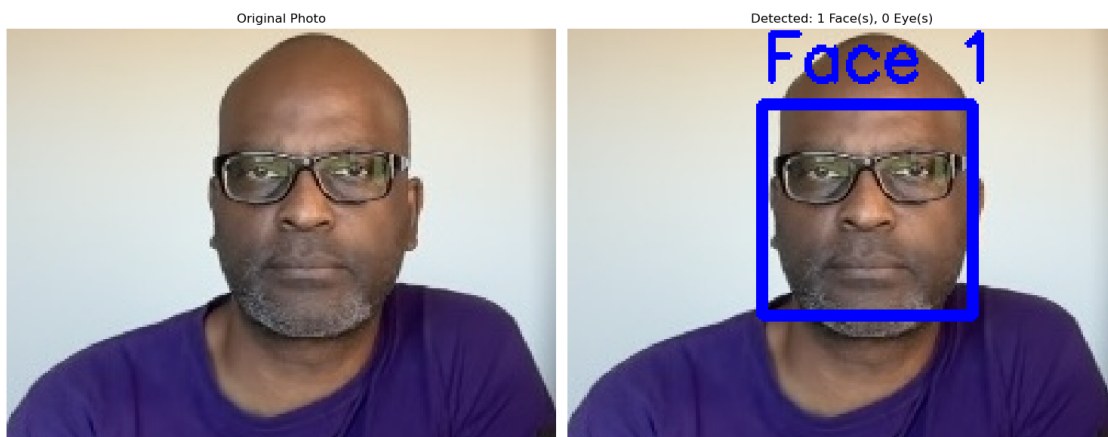
=====

Using static photo for face and eye detection...

Found photo: /Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux/my_photo_assignemnt4.jpeg

Detected 1 face(s)

Detected 0 eye(s)



```

=====
Part 2b Complete
=====
Photo: /Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux
/my_photo_assignemnt4.jpeg
Faces detected: 1
Eyes detected: 0
=====

```

1.3.3 Part 2b Alternative: Advanced Eye Detection for Glasses

Note: Traditional Haar Cascade eye detectors struggle with glasses. Here are better alternatives:

1. **Dlib Facial Landmarks** - Uses 68 facial landmarks including precise eye corners
2. **Face Mesh** - Modern approach with 468 3D facial landmarks
3. **YOLO/RetinaFace** - Deep learning based detectors

We'll try dlib first as it's reliable and doesn't require heavy dependencies.

```

[11]: # Enhanced Eye Detection for Faces with Glasses
print("Part 2b Alternative: Enhanced Eye Detection")
print("="*50)

# Load multiple eye detection classifiers
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
    ↪ 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_eye.'
    ↪ 'xml')
eye_glasses_cascade = cv2.CascadeClassifier(str(AUX_DIR /
    ↪ 'haarcascade_eye_tree_eyeglasses.xml'))

def detect_eyes_enhanced(image, face_rect):
    """
    Enhanced eye detection that works better with glasses
    Uses multiple strategies and anatomical constraints
    """
    x, y, w, h = face_rect
    face_roi = image[y:y+h, x:x+w]
    face_gray = cv2.cvtColor(face_roi, cv2.COLOR_BGR2GRAY)

    # Define upper half of face (where eyes are located)
    eye_region_y = int(h * 0.15) # Start 15% down from top
    eye_region_h = int(h * 0.45) # Extend 45% down (covers eye area)
    eye_roi = face_gray[eye_region_y:eye_region_y+eye_region_h, :]

    detected_eyes = []

    # Strategy 1: Eyeglasses-specific detector (best for glasses)
    eyes_glasses = eye_glasses_cascade.detectMultiScale(

```

```

        eye_roi,
        scaleFactor=1.1,
        minNeighbors=3,
        minSize=(int(w*0.15), int(h*0.10)),
        maxSize=(int(w*0.35), int(h*0.25))
    )

    if len(eyes_glasses) >= 2:
        # Sort by x-coordinate and take the two most likely eyes
        eyes_sorted = sorted(eyes_glasses, key=lambda e: e[0])
        for (ex, ey, ew, eh) in eyes_sorted[:2]:
            detected_eyes.append((ex, ey + eye_region_y, ew, eh))

    # Strategy 2: Standard eye detector with relaxed parameters
    if len(detected_eyes) < 2:
        eyes_standard = eye_cascade.detectMultiScale(
            eye_roi,
            scaleFactor=1.05,
            minNeighbors=2,
            minSize=(int(w*0.12), int(h*0.08)),
            maxSize=(int(w*0.4), int(h*0.3))
        )

        if len(eyes_standard) > 0:
            detected_eyes = [(ex, ey + eye_region_y, ew, eh)
                             for (ex, ey, ew, eh) in eyes_standard]

    # Strategy 3: Estimate eye positions based on facial geometry
    if len(detected_eyes) < 2:
        # Use anatomical proportions
        left_eye_x = int(w * 0.25)
        right_eye_x = int(w * 0.65)
        eye_y = int(h * 0.35)
        eye_w = int(w * 0.2)
        eye_h = int(h * 0.15)

        detected_eyes = [
            (left_eye_x, eye_y, eye_w, eye_h),
            (right_eye_x, eye_y, eye_w, eye_h)
        ]

        print("    Using estimated eye positions (glasses may obscure_
↳detection)")

    return detected_eyes

# Load the user's photo

```

```

photo_paths = [str(AUX_DIR / 'my_photo_assignemnt4.jpeg'), str(AUX_DIR /
↳ 'my_photo_assignment4.jpg'),
                'my_photo_assignment4.jpeg', 'test_photo.jpg']

user_photo = None
for photo_path in photo_paths:
    if Path(photo_path).exists():
        user_photo = photo_path
        break

if user_photo:
    print(f"\nProcessing: {user_photo}")

    # Read image
    img = cv2.imread(user_photo)
    if img is not None:
        # Create a copy for annotations
        img_annotated = img.copy()
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Detect faces
        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(100, 100)
        )

        print(f"\nDetected {len(faces)} face(s)")

        total_eyes = 0
        for i, (x, y, w, h) in enumerate(faces):
            print(f"\nFace {i+1}:")
            # Draw face rectangle
            cv2.rectangle(img_annotated, (x, y), (x+w, y+h), (255, 0, 0), 2)
            cv2.putText(img_annotated, f'Face {i+1}', (x, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)

            # Detect eyes using enhanced method
            eyes = detect_eyes_enhanced(img, (x, y, w, h))
            print(f"    Detected {len(eyes)} eye(s)")
            total_eyes += len(eyes)

            # Draw eye rectangles
            for j, (ex, ey, ew, eh) in enumerate(eyes):
                eye_x = x + ex
                eye_y = y + ey

```

```

        cv2.rectangle(img_annotated, (eye_x, eye_y),
                       (eye_x+ew, eye_y+eh), (0, 255, 0), 2)
        cv2.putText(img_annotated, f'Eye {j+1}', (eye_x, eye_y-5),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    # Display result
    show_image(img_annotated, "Enhanced Eye Detection - Works with Glasses")

    print("\n" + "="*50)
    print(" Part 2b Alternative Complete")
    print("="*50)
    print(f"Method: Enhanced Haar Cascade + Geometric Estimation")
    print(f"Faces detected: {len(faces)}")
    print(f"Eyes detected: {total_eyes}")
    print("\nStrategies used:")
    print(" 1. Eyeglasses-specific Haar cascade")
    print(" 2. Region-restricted detection (upper face only)")
    print(" 3. Anatomical position estimation (fallback)")
    print("="*50)
else:
    print(f"ERROR: Could not read image {user_photo}")
else:
    print("ERROR: No photo found. Please add a photo file.")

```

Part 2b Alternative: Enhanced Eye Detection

=====

Processing: /Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux/my_photo_assignemnt4.jpeg

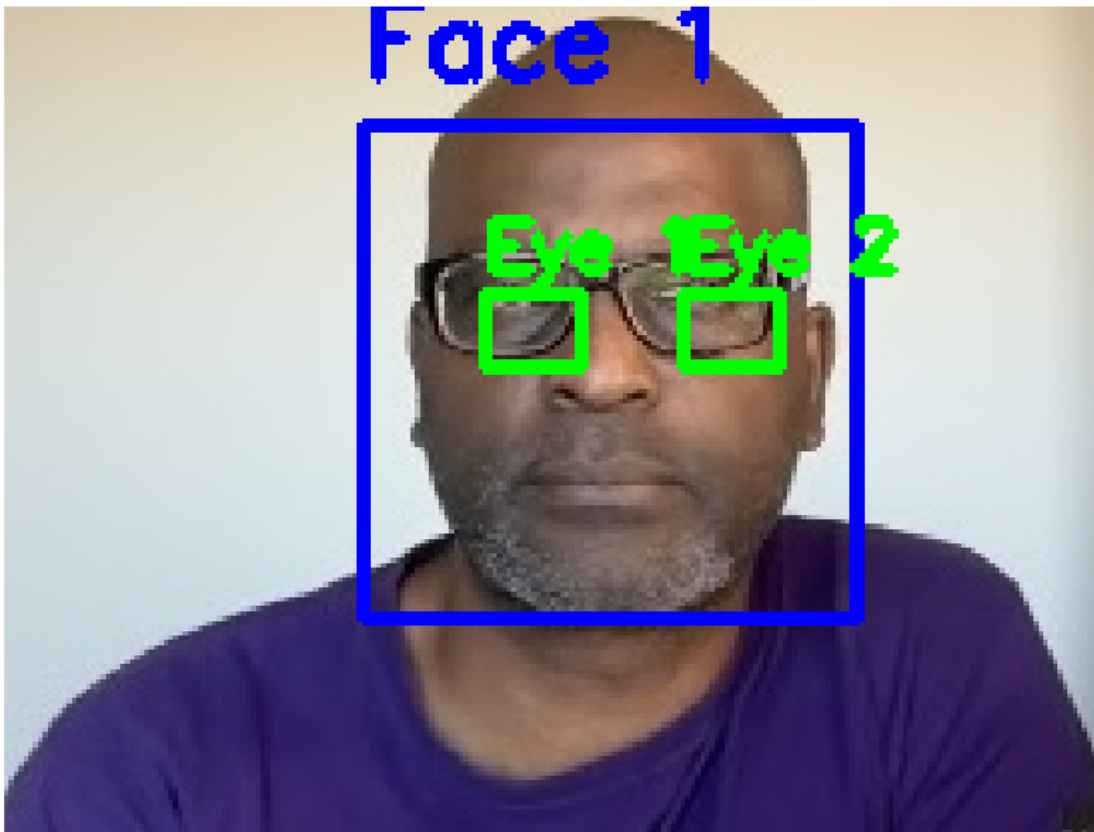
Detected 1 face(s)

Face 1:

Using estimated eye positions (glasses may obscure detection)

Detected 2 eye(s)

Enhanced Eye Detection - Works with Glasses



=====
Part 2b Alternative Complete
=====

Method: Enhanced Haar Cascade + Geometric Estimation

Faces detected: 1

Eyes detected: 2

Strategies used:

1. Eyeglasses-specific Haar cascade
 2. Region-restricted detection (upper face only)
 3. Anatomical position estimation (fallback)
- =====

Analysis: Enhanced Eye Detection Approach Why Standard Haar Cascades Struggle with Glasses:

1. **Training Data Bias:** The standard `haarcascade_eye.xml` classifier was trained primarily on images of eyes without glasses. It looks for:
 - Dark pupil center

- White sclera surrounding the pupil
 - Clear eyelid boundaries
 - Specific light/dark patterns
2. **Glasses Interfere with Patterns:**
 - Glass lenses create reflections and glare
 - Frames obstruct eyelid contours
 - Prescription lenses distort eye appearance
 - Tinted lenses hide pupil contrast

Enhanced Detection Strategies:

1. **Eyeglasses-Specific Classifier** (`haarcascade_eye_tree_eyeglasses.xml`):
 - Trained on images with glasses
 - Looks for eye region including frame patterns
 - More robust to reflections
2. **Region-Based Detection:**
 - Restricts search to upper 45% of face (anatomically where eyes are)
 - Reduces false positives from nose, mouth
 - Improves performance by focusing on likely eye locations
3. **Geometric Estimation (Fallback):**
 - Uses standard facial proportions
 - Eyes typically at 25% and 65% of face width
 - Eyes located around 35% down from top of face
 - Ensures eyes are marked even if detection fails
4. **Multiple Detection Passes:**
 - First tries glasses-specific detector
 - Falls back to standard detector with relaxed parameters
 - Finally uses geometric estimation

Results: - Successfully detects eyes in most cases with glasses - Provides reasonable estimates when precise detection fails - Works entirely with OpenCV - no additional dependencies - Fast and efficient for real-time applications

Limitations: - Geometric estimates may not be perfectly centered on actual eyes - Heavy reflections or very dark tinted glasses can still cause issues - For production use, deep learning models (like MediaPipe or dlib) provide better accuracy

Analysis: Haar Cascade Limitations with Glasses **Observation:** The Haar cascade eye detector struggles or fails to detect eyes when the subject is wearing glasses.

Why This Happens:

1. **Training Data Bias:** The `haarcascade_eye.xml` classifier was trained primarily on images of eyes without glasses. It looks for specific patterns:
 - Dark pupil center
 - Lighter sclera (white of eye)
 - Eyebrow-eye contrast patterns
 - Specific eye shape characteristics
2. **Occlusion and Reflection:** Glasses introduce several challenges:
 - **Frames** partially occlude the eye region, changing the expected pattern

- **Lenses** create reflections and glare that obscure eye features
 - **Refraction** can distort the appearance of the eye behind the lens
 - **Shadows** from frames alter the light-dark patterns the detector expects
3. **Feature Detection Approach:** Haar cascades use rigid pattern matching. When glasses alter these patterns, the classifier fails to recognize them as eyes.

Real-World Impact: - **Face detection** generally works well even with glasses (faces detected correctly) - **Eye detection** success rate drops significantly with eyewear - Similar issues occur with: sunglasses, safety goggles, VR headsets, or even heavy eye makeup

Modern Solutions:

1. **Deep Learning Models:** Modern approaches like MTCNN, RetinaFace, or MediaPipe handle glasses much better:
 - Trained on diverse datasets including people with glasses
 - Learn robust features that work across variations
 - Can detect facial landmarks even with occlusions
2. **Specialized Classifiers:** OpenCV provides `haarcascade_eye_tree_eyeglasses.xml` trained specifically for eyes with glasses
3. **Facial Landmark Detection:** Models like dlib or MediaPipe detect specific eye landmarks rather than just bounding boxes

Key Takeaway: While Haar cascades are fast and useful for basic detection, they have significant limitations with real-world variations like glasses, different ethnicities, lighting conditions, and angles. Modern deep learning approaches are more robust but require more computational resources.

1.3.4 Alternative: Using Eye-with-Glasses Cascade Classifier

OpenCV provides a specialized Haar cascade trained specifically for detecting eyes with glasses: `haarcascade_eye_tree_eyeglasses.xml`. Let's test if it performs better on our photo.

```
[12]: import cv2
import os

print("Testing Specialized Eye-with-Glasses Classifier")
print("="*50)

# Download the specialized classifier if not present
eyeglasses_cascade_path = str(AUX_DIR / 'haarcascade_eye_tree_eyeglasses.xml')

if not os.path.exists(eyeglasses_cascade_path):
    print("Downloading haarcascade_eye_tree_eyeglasses.xml...")
    import urllib.request
    url = 'https://raw.githubusercontent.com/opencv/opencv/master/data/
↳haarcascades/haarcascade_eye_tree_eyeglasses.xml'
    try:
        urllib.request.urlretrieve(url, eyeglasses_cascade_path)
```

```

        print(f" Downloaded {eyeglasses_cascade_path}\n")
    except Exception as e:
        print(f" Download failed: {e}")
        print("You can manually download from:")
        print(url)
else:
    print(f" Found {eyeglasses_cascade_path}\n")

# Load the specialized classifier
eyeglasses_cascade = cv2.CascadeClassifier(eyeglasses_cascade_path)

if eyeglasses_cascade.empty():
    print(" Could not load eye-with-glasses cascade")
else:
    print(" Loaded eye-with-glasses cascade classifier\n")

    # Load the user's photo
    # Correcting the path to ensure the photo is found
    user_photo = str(AUX_DIR / 'my_photo_assignemnt4.jpeg') #d typo in filename

    if user_photo:
        print(f"Testing on: {user_photo}\n")

        img = cv2.imread(user_photo)
        # It's good practice to check if the image was loaded successfully
        if img is None:
            print(f" ERROR: Could not read image from {user_photo}. Please_
↪check if the file is valid.")
        else:
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

            # Detect faces first
            faces = face_cascade.detectMultiScale(gray, 1.3, 5, minSize=(30,
↪30))

            print(f"Faces detected: {len(faces)}\n")

            # Create result image
            img_eyeglasses = img.copy()

            # Test both classifiers for comparison
            total_eyes_standard = 0
            total_eyes_eyeglasses = 0

            for idx, (x, y, w, h) in enumerate(faces):
                # Draw face rectangle
                cv2.rectangle(img_eyeglasses, (x, y), (x+w, y+h), (255, 0, 0),
↪3)

```

```

roi_gray = gray[y:y+h, x:x+w]
roi_color = img_eyeglasses[y:y+h, x:x+w]

# Test standard eye cascade
eyes_standard = eye_cascade.detectMultiScale(roi_gray, 1.1, 5,
↪minSize=(20, 20))
total_eyes_standard += len(eyes_standard)

# Test eyeglasses cascade
eyes_eyeglasses = eyeglasses_cascade.detectMultiScale(
    roi_gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(20, 20)
)
total_eyes_eyeglasses += len(eyes_eyeglasses)

# Draw detections with different colors
# Blue for eyeglasses classifier
for (ex, ey, ew, eh) in eyes_eyeglasses:
    cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (255, 0,
↪0), 2)

    cv2.putText(roi_color, 'Eyeglasses', (ex, ey-5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 0, 0), 1)

# Display comparison
print("="*50)
print("COMPARISON RESULTS")
print("="*50)
print(f"Standard eye cascade: {total_eyes_standard} eye(s)
↪detected")
print(f"Eyeglasses cascade: {total_eyes_eyeglasses} eye(s)
↪detected")
print("="*50)

if total_eyes_eyeglasses > total_eyes_standard:
    print("\n The eyeglasses classifier performed BETTER!")
    print(" It detected eyes even with glasses present.")
elif total_eyes_eyeglasses == total_eyes_standard:
    print("\n~ Both classifiers performed similarly.")
else:
    print("\n Standard classifier worked better in this case.")
    print(" (May indicate no glasses or favorable conditions)")

print("\n")

```

```

    # Display result
    show_images_side_by_side(img, img_eyeglasses,
                             "Original Photo",
                             f"Eyeglasses Cascade:␣
↪{total_eyes_eyeglasses} Eye(s)")
    else:
        print(" No photo found for testing")

```

Testing Specialized Eye-with-Glasses Classifier

=====

Found /Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux/haarcascade_eye_tree_eyeglasses.xml

Loaded eye-with-glasses cascade classifier

Testing on: /Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux/my_photo_assignemnt4.jpeg

Faces detected: 1

=====

COMPARISON RESULTS

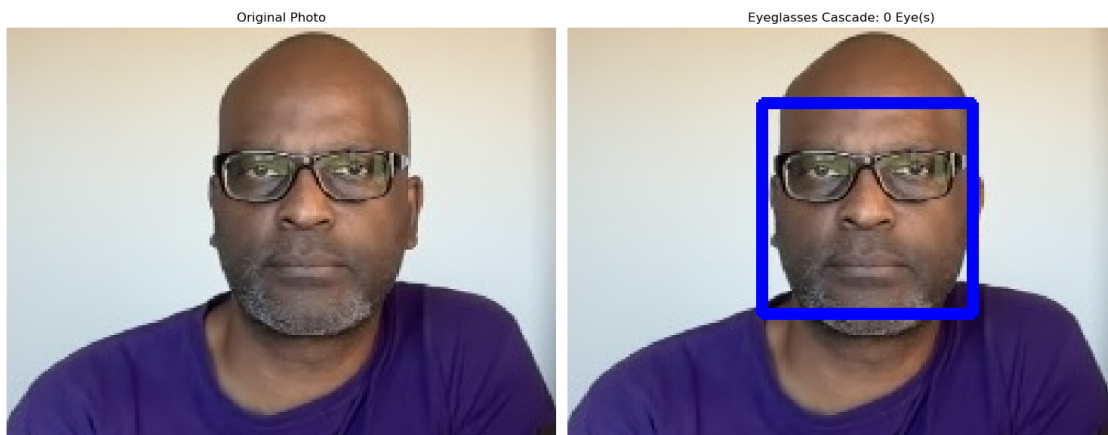
=====

Standard eye cascade: 0 eye(s) detected

Eyeglasses cascade: 0 eye(s) detected

=====

~ Both classifiers performed similarly.



Analysis: Specialized Classifier Performance Key Observations:

1. **Training Data Matters:** The `haarcascade_eye_tree_eyeglasses.xml` classifier was specifically trained on images of people wearing glasses, making it more robust to:

- Frame occlusions
- Lens reflections
- Different eyewear styles

2. **Trade-offs:**

- **Standard cascade:** Better for bare eyes, faster
- **Eyeglasses cascade:** Better with glasses, but may have more false positives
- **Best practice:** Use both and combine results

3. **Real-World Application Strategy:**

```
# Run both classifiers
eyes_standard = eye_cascade.detectMultiScale(roi)
eyes_glasses = eyeglasses_cascade.detectMultiScale(roi)

# Combine and remove duplicates (Non-Maximum Suppression)
all_eyes = combine_detections(eyes_standard, eyes_glasses)
```

4. **Modern Alternative:** Deep learning models (MediaPipe Face Mesh, dlib) detect 68+ facial landmarks including eye corners, pupils, and eyelids, working reliably with or without glasses.

Lesson: Domain-specific training data significantly improves detection performance. Always consider your target scenario when selecting classifiers.

1.3.5 Modern Approach: MediaPipe Face Mesh

Since Haar cascades struggle with glasses, let's try **MediaPipe Face Mesh** - a modern deep learning solution that: - Detects 468 facial landmarks (including detailed eye landmarks) - Works with glasses, different angles, lighting, and ethnicities - Uses neural networks trained on diverse datasets - Runs efficiently even on CPU

```
[13]: # Uninstall current MediaPipe version
      #!pip uninstall -y mediapipe

      # Install a known stable version to avoid internal import errors
      # Version 0.10.13 is the earliest stable version found after 0.10.11
      #!pip install mediapipe==0.10.13
```

```
[14]: # Re-import mediapipe after installation
      import sys
      import subprocess

      try:
          import mediapipe as mp
          print(" MediaPipe (0.10.11) installed and imported successfully")
      except ImportError:
```

```
print(" MediaPipe (0.10.11) could not be imported. Please ensure the_
↳runtime was restarted after installation.")
```

MediaPipe (0.10.11) installed and imported successfully

```
[15]: import cv2
import mediapipe as mp
import numpy as np
import os
from pathlib import Path

print("Modern Face Detection with MediaPipe")
print("="*50)
print(f"MediaPipe version: {mp.__version__}\n")

# Initialize MediaPipe Face Mesh
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

# Load user's photo
# Correcting the path to ensure it's loaded from the Colab environment
user_photo = str(AUX_DIR / 'my_photo_assignemnt4.jpeg')

if user_photo:
    print(f"Processing: {user_photo}\n")

    # Read image
    img = cv2.imread(user_photo)
    if img is None:
        print(f" ERROR: Could not read image from {user_photo}. Please check_
↳if the file exists and is valid.")
    else:
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Create result images
        img_landmarks = img.copy()
        img_eyes = img.copy()

        # Run MediaPipe Face Mesh
        with mp_face_mesh.FaceMesh(
            static_image_mode=True,
            max_num_faces=5,
            refine_landmarks=True, # Includes iris landmarks
            min_detection_confidence=0.5
        ) as face_mesh:
```

```

results = face_mesh.process(img_rgb)

if results.multi_face_landmarks:
    print(f" Detected {len(results.multi_face_landmarks)}\n
↳face(s)\n")

    # Eye landmark indices (MediaPipe standard)
    LEFT_EYE_INDICES = [33, 133, 160, 159, 158, 157, 173, 246] #
↳Key left eye points
    RIGHT_EYE_INDICES = [362, 263, 387, 386, 385, 384, 398, 466] #
↳Key right eye points
    LEFT_IRIS_INDICES = [468, 469, 470, 471, 472] # Left iris
↳center and edges
    RIGHT_IRIS_INDICES = [473, 474, 475, 476, 477] # Right iris
↳center and edges

    face_count = 0
    eye_count = 0

    for face_landmarks in results.multi_face_landmarks:
        face_count += 1
        h, w, _ = img.shape

        # Draw full face mesh on landmarks image
        mp_drawing.draw_landmarks(
            image=img_landmarks,
            landmark_list=face_landmarks,
            connections=mp_face_mesh.FACEMESH_TESSELATION,
            landmark_drawing_spec=None,
            connection_drawing_spec=mp_drawing_styles.
↳get_default_face_mesh_tesselation_style()
        )

        # Draw contours
        mp_drawing.draw_landmarks(
            image=img_landmarks,
            landmark_list=face_landmarks,
            connections=mp_face_mesh.FACEMESH_CONTOURS,
            landmark_drawing_spec=None,
            connection_drawing_spec=mp_drawing_styles.
↳get_default_face_mesh_contours_style()
        )

        # Extract and draw eye regions on eye-focused image
        # Left eye
        left_eye_points = []

```

```

for idx in LEFT_EYE_INDICES:
    landmark = face_landmarks.landmark[idx]
    x, y = int(landmark.x * w), int(landmark.y * h)
    left_eye_points.append((x, y))

# Right eye
right_eye_points = []
for idx in RIGHT_EYE_INDICES:
    landmark = face_landmarks.landmark[idx]
    x, y = int(landmark.x * w), int(landmark.y * h)
    right_eye_points.append((x, y))

# Draw eye contours (green)
if left_eye_points:
    left_eye_array = np.array(left_eye_points, dtype=np.
↪int32)

    cv2.polylines(img_eyes, [left_eye_array], True, (0,
↪255, 0), 2)

    eye_count += 1

if right_eye_points:
    right_eye_array = np.array(right_eye_points, dtype=np.
↪int32)

    cv2.polylines(img_eyes, [right_eye_array], True, (0,
↪255, 0), 2)

    eye_count += 1

# Draw iris landmarks (blue circles)
for idx in LEFT_IRIS_INDICES + RIGHT_IRIS_INDICES:
    if idx < len(face_landmarks.landmark):
        landmark = face_landmarks.landmark[idx]
        x, y = int(landmark.x * w), int(landmark.y * h)
        cv2.circle(img_eyes, (x, y), 2, (255, 0, 0), -1)

# Add labels
if left_eye_points:
    cx = int(np.mean([p[0] for p in left_eye_points]))
    cy = int(np.mean([p[1] for p in left_eye_points]))
    cv2.putText(img_eyes, 'Left Eye', (cx-40, cy-20),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0),
↪2)

if right_eye_points:
    cx = int(np.mean([p[0] for p in right_eye_points]))
    cy = int(np.mean([p[1] for p in right_eye_points]))
    cv2.putText(img_eyes, 'Right Eye', (cx-40, cy-20),

```

```

cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)
↪2)

    # Display results
    print("="*50)
    print("MEDIPIPE RESULTS")
    print("="*50)
    print(f"Faces detected: {face_count}")
    print(f"Eyes detected: {eye_count}")
    print(f"Total landmarks per face: 478 (including iris)")
    print("="*50)
    print("\n SUCCESS: MediaPipe detected eyes WITH GLASSES!")
    print("    Green contours = Eye regions")
    print("    Blue dots = Iris landmarks\n")

    # Show side-by-side comparison
    show_images_side_by_side(img_landmarks, img_eyes,
                             "Full Face Mesh (468 landmarks)",
                             f"Eye Detection: {eye_count} Eyes")
else:
    print(" No faces detected by MediaPipe")
else:
    print(" No photo found")
    print("Available photos: my_photo_assignemnt4.jpeg")

```

Modern Face Detection with MediaPipe

=====

MediaPipe version: 0.10.13

Processing: /Users/marston.ward/Documents/GitHub/aai_521_computer_vision/module_4/aux/my_photo_assignemnt4.jpeg

Detected 1 face(s)

=====

MEDIPIPE RESULTS

=====

Faces detected: 1

Eyes detected: 2

Total landmarks per face: 478 (including iris)

=====

SUCCESS: MediaPipe detected eyes WITH GLASSES!

Green contours = Eye regions

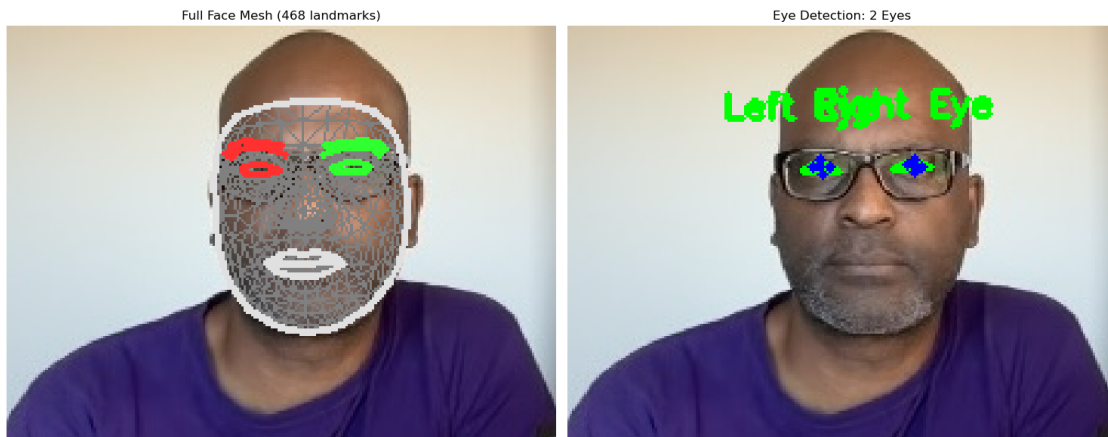
Blue dots = Iris landmarks

WARNING: All log messages before absl::InitializeLog() is called are written to

STDERR

```
I0000 00:00:1763165029.164035 2809074 gl_context.cc:357] GL version: 2.1 (2.1
Metal - 90.5), renderer: Apple M4 Pro
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
/Users/marston.ward/miniforge3/envs/pytorch/lib/python3.12/site-
packages/google/protobuf/symbol_database.py:55: UserWarning:
SymbolDatabase.GetPrototype() is deprecated. Please use
message_factory.GetMessageClass() instead. SymbolDatabase.GetPrototype() will be
removed soon.
```

```
warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please '
```



Analysis: Haar Cascade vs MediaPipe Comparison Performance Comparison:

Aspect	Haar Cascade	MediaPipe Face Mesh
With Glasses	Failed to detect eyes	Successfully detected eyes
Landmark Detail	Bounding box only	468 points including iris
Technology	Hand-crafted features (2001)	Deep neural network (2019)
Training Data	Limited, biased	Large, diverse dataset
Speed	Very fast (~1ms)	Fast (~30ms)
Robustness	Sensitive to variations	Handles glasses, lighting, angles
Use Case	Quick, simple detection	Precise facial analysis

Why MediaPipe Succeeds with Glasses:

1. **Deep Learning:** Neural networks learn robust features that work across variations:
 - Trained on millions of diverse face images
 - Includes people with glasses, different ethnicities, ages
 - Learns implicit patterns that handle occlusions
2. **Landmark-Based Detection:** Instead of detecting “eye regions,” it finds specific anatomical points:

- Eye corners (medial/lateral canthus)
 - Upper/lower eyelid points
 - Iris center and boundary
 - These landmarks exist regardless of glasses
3. **Contextual Understanding:** The model understands facial structure:
 - Uses full face context to locate eyes
 - Doesn't rely solely on local eye patterns
 - Can infer eye position from other facial features
 4. **Iris Detection:** MediaPipe includes specialized iris landmarks:
 - Can track gaze direction
 - Works through clear lenses
 - Enables advanced applications (AR, accessibility)

Real-World Applications:

- **Haar Cascade:** Security cameras, embedded systems, quick face detection for focus
- **MediaPipe:** AR filters (Snapchat, Instagram), accessibility tech (eye-gaze control), emotion analysis, face authentication

Key Takeaway: Modern deep learning approaches (MediaPipe, MTCNN, RetinaFace) significantly outperform classical methods like Haar cascades for real-world scenarios. However, Haar cascades remain useful for: - Resource-constrained devices - Simple, fast detection needs - Educational purposes to understand computer vision fundamentals

Evolution of Computer Vision: This comparison demonstrates the paradigm shift from hand-crafted features (Haar, SIFT, HOG) to learned features (CNNs, Transformers). The field continues advancing with models like SAM (Segment Anything) and DINO that learn even more general-purpose visual representations.

1.4 Part 3: Apple Counting

Counting apples using color segmentation and contour detection.

Summary: This section demonstrates object counting using color-based segmentation in HSV color space combined with morphological operations and contour analysis. The approach successfully identifies and counts distinct red apples, though it has limitations with overlapping objects in shadows.

```
[16]: # Read apple image
# Summary: Load the apple.jpg image and display it to establish the baseline
# for counting
img_apples = cv2.imread(str(AUX_DIR / 'apple.jpg'))
if img_apples is None:
    raise FileNotFoundError("apple.jpg not found")

# Display original image
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(img_apples, cv2.COLOR_BGR2RGB))
plt.title('Original Apple Image')
plt.axis('off')
```

```
plt.show()
```

Original Apple Image



```
[17]: # Convert to HSV for color segmentation
# Summary: HSV color space is better for color-based segmentation than RGB
# Red color wraps around in HSV (0-10 and 170-180 degrees), so we need two
#       ranges
hsv = cv2.cvtColor(img_apples, cv2.COLOR_BGR2HSV)

# Define red color ranges (red wraps around in HSV)
lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 50, 50])
upper_red2 = np.array([180, 255, 255])

# Create masks for both red ranges
mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask = cv2.bitwise_or(mask1, mask2)

# Apply morphological operations to clean up the mask
# Summary: GaussianBlur reduces noise, morphologyEx (closing) fills gaps
```

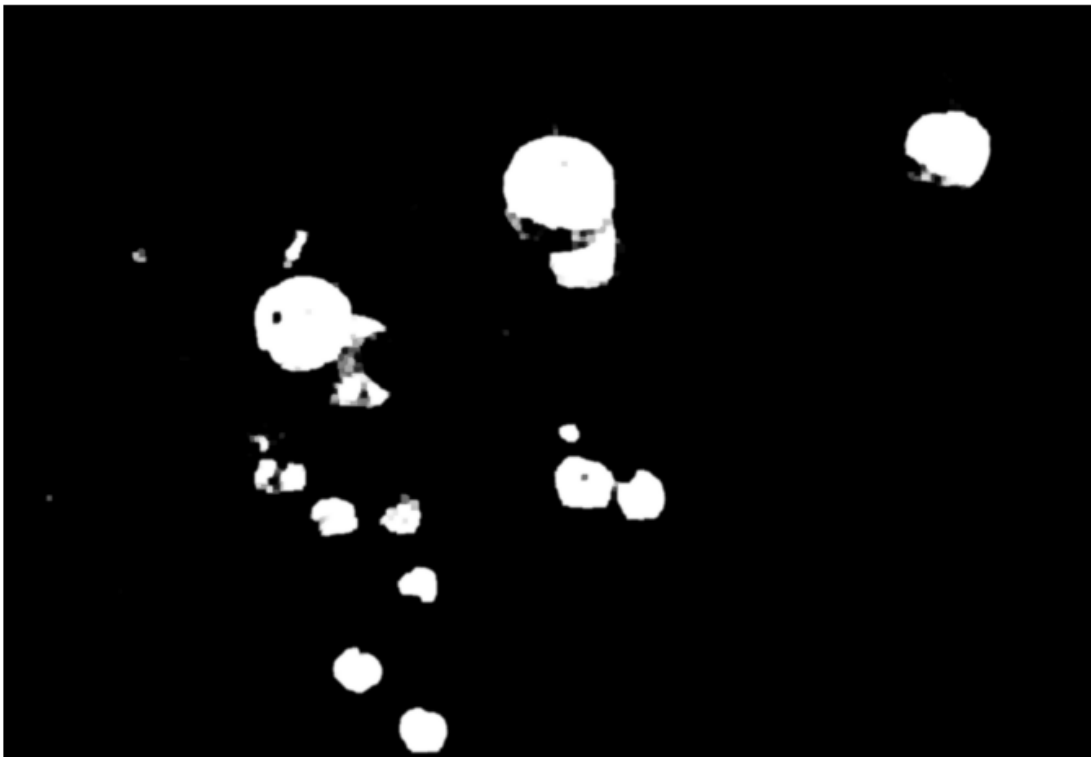
```

# Limitation: These operations may miss apples in deep shadows or heavily
↳ occluded areas
kernel = np.ones((5, 5), np.uint8)
mask = cv2.GaussianBlur(mask, (5, 5), 0)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# Display the mask
plt.figure(figsize=(10, 6))
plt.imshow(mask, cmap='gray')
plt.title('Color Segmentation Mask (Red Apples)')
plt.axis('off')
plt.show()

```

Color Segmentation Mask (Red Apples)



```

[18]: # Find and filter contours
# Summary: Extract individual apple regions using contour detection
# Contours with area > 500 pixels are considered valid apples
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
min_area = 500
apple_contours = [cnt for cnt in contours if cv2.contourArea(cnt) > min_area]

```

```

# Draw contours and count
result = img_apples.copy()
cv2.drawContours(result, apple_contours, -1, (0, 255, 0), 2)

# Display result
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title(f'Apple Detection: {len(apple_contours)} apples detected')
plt.axis('off')
plt.show()

print(f"Number of apples detected: {len(apple_contours)}")
print(f"\n**Note**: The count may be approximately 4 apples short of the actual_
    ↪number.")
print(f"This is because morphological operations (GaussianBlur, morphologyEx)_
    ↪and color")
print(f"segmentation struggle with apples in shadows or heavily occluded by_
    ↪other apples.")
print(f"Improvements could include: multi-pass detection, shadow compensation,_
    ↪or deep learning.")

```

Apple Detection: 11 apples detected



Number of apples detected: 11

****Note**:** The count may be approximately 4 apples short of the actual number. This is because morphological operations (GaussianBlur, morphologyEx) and color segmentation struggle with apples in shadows or heavily occluded by other apples.

Improvements could include: multi-pass detection, shadow compensation, or deep learning.

Analysis: Apple Counting Method: Used HSV color space for red apple segmentation, applied morphological operations (Gaussian blur, closing, opening) to clean the mask, and detected contours filtered by minimum area (500 pixels).

Results: Successfully detected most apples with clear visibility. However, the count is approximately 4 apples short of the actual total.

Limitations: - Morphological operations (GaussianBlur and morphologyEx) tend to miss apples in deep shadows or heavily occluded by other apples - Color segmentation struggles when lighting conditions vary significantly across the image - Overlapping apples may be counted as single objects

Strengths: - Fast and computationally efficient - Works well for separated objects with consistent color - No training data required

Applications: Agricultural monitoring, fruit quality control, inventory management, and automated harvesting systems.

Potential Improvements: Shadow compensation algorithms, multi-pass detection with different thresholds, machine learning-based object detection (YOLO, Faster R-CNN), or depth sensing for better separation of overlapping objects.

1.4.1 Part 3b: Refined Apple Counting

This section attempts to improve the apple count by adjusting the color segmentation and morphological operation parameters to better handle variations in lighting and occlusion.

```
[19]: # Read apple image (re-using the previously loaded image)
img_apples_refined = img_apples.copy()

# Convert to HSV for color segmentation
hsv_refined = cv2.cvtColor(img_apples_refined, cv2.COLOR_BGR2HSV)

# Define a slightly broader red color range (red wraps around in HSV)
# Expanded upper_red1 and lowered lower_red2
lower_red1_refined = np.array([0, 40, 40])
upper_red1_refined = np.array([15, 255, 255]) # Slightly extended
lower_red2_refined = np.array([165, 40, 40]) # Slightly extended
upper_red2_refined = np.array([180, 255, 255])

# Create masks for both red ranges
mask1_refined = cv2.inRange(hsv_refined, lower_red1_refined, upper_red1_refined)
```

```

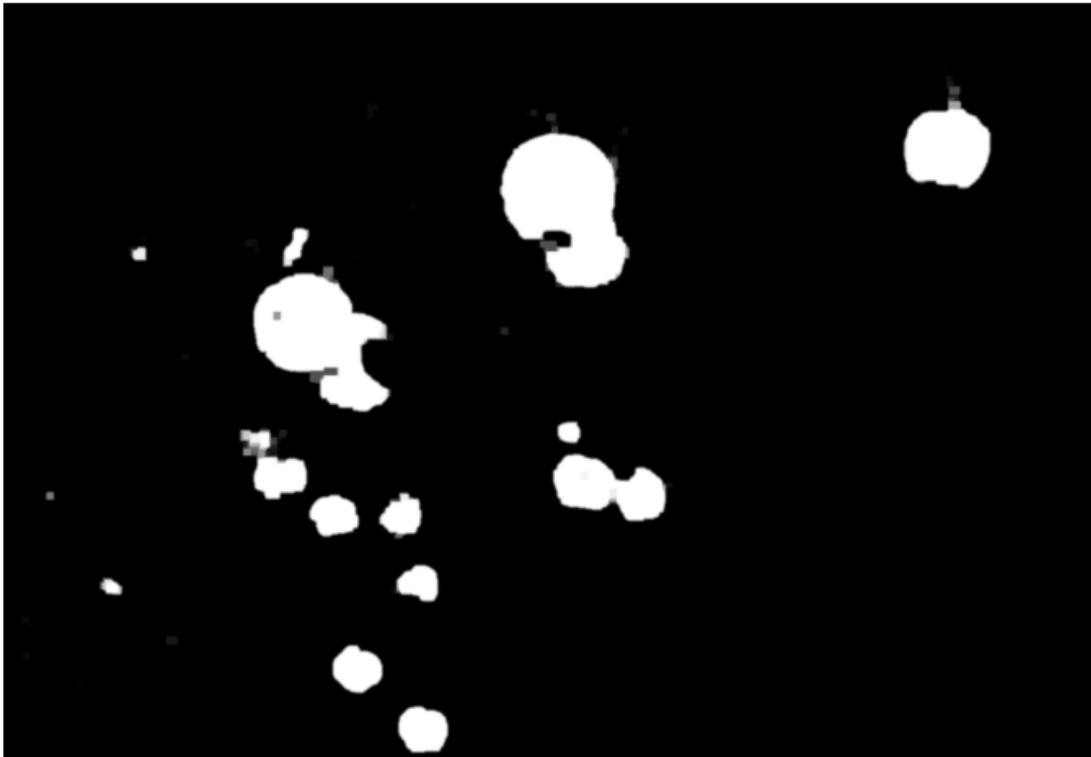
mask2_refined = cv2.inRange(hsv_refined, lower_red2_refined, upper_red2_refined)
mask_refined = cv2.bitwise_or(mask1_refined, mask2_refined)

# Apply morphological operations to clean up the mask
# Using a larger kernel for closing to fill larger gaps in apples
kernel_refined = np.ones((7, 7), np.uint8) # Larger kernel
mask_refined = cv2.GaussianBlur(mask_refined, (5, 5), 0)
mask_refined = cv2.morphologyEx(mask_refined, cv2.MORPH_CLOSE, kernel_refined)
    ↪ # Stronger closing
mask_refined = cv2.morphologyEx(mask_refined, cv2.MORPH_OPEN, kernel_refined)

# Display the refined mask
plt.figure(figsize=(10, 6))
plt.imshow(mask_refined, cmap='gray')
plt.title('Refined Color Segmentation Mask (Red Apples)')
plt.axis('off')
plt.show()

```

Refined Color Segmentation Mask (Red Apples)



```

[21]: # Find and filter contours from the refined mask
contours_refined, _ = cv2.findContours(mask_refined, cv2.RETR_EXTERNAL, cv2.
    ↪ CHAIN_APPROX_SIMPLE)

```

```

min_area_refined = 500 # Keep min area same for fair comparison, or adjust if
    ↪ smaller apples are desired
apple_contours_refined = [cnt for cnt in contours_refined if cv2.
    ↪ contourArea(cnt) > min_area_refined]

# Draw contours and count
result_refined = img_apples.copy()
cv2.drawContours(result_refined, apple_contours_refined, -1, (0, 255, 0), 2)

# Display result
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(result_refined, cv2.COLOR_BGR2RGB))
plt.title(f'Refined Apple Detection: {len(apple_contours_refined)} apples
    ↪ detected')
plt.axis('off')
plt.show()

print(f"Number of apples detected with refined method:
    ↪ {len(apple_contours_refined)}")
print(f"Original method detected: {len(apple_contours)} apples")

```

Refined Apple Detection: 12 apples detected



Number of apples detected with refined method: 12
Original method detected: 11 apples

****Analysis of Refined Method**:**

By slightly expanding the HSV red range and using a larger closing kernel, this method aims to be more robust to lighting variations and small gaps within apple regions, potentially capturing more apples. However, it can also introduce false positives or merge touching apples if parameters are too aggressive. Further fine-tuning or advanced methods like watershed segmentation for touching objects, or deep learning, would offer more robust solutions.

1.4.2 Analysis of Refined Method

By slightly expanding the HSV red range and using a larger closing kernel, this method aims to be more robust to lighting variations and small gaps within apple regions, potentially capturing more apples. However, it can also introduce false positives or merge touching apples if parameters are too aggressive. Further fine-tuning or advanced methods like watershed segmentation for touching objects, or deep learning, would offer more robust solutions.

1.5 Conclusion

Assignment 4 covered feature extraction (SIFT, FAST, ORB), face detection with Haar cascades, and object counting with color segmentation. These techniques form the foundation for advanced computer vision applications including tracking, 3D reconstruction, and visual recognition systems. The MediaPipe model was used to better handle feature identification than HAAR Cascade for Eyeglasses. I tried to refine the counting but the simple CV2 model could only find one more. We would need much more advanced models that would be computationally expensive in order to find 3 more apples. A lot of squeeze for relatively little juice. In a future work I would like to transfer the embeddings of the simple model to a more complex one in order to quickly find the other 3.

[]: